Unit-5
Data Visualization

Presented By:

Sindhu Patchigolla

Asst.Professor©

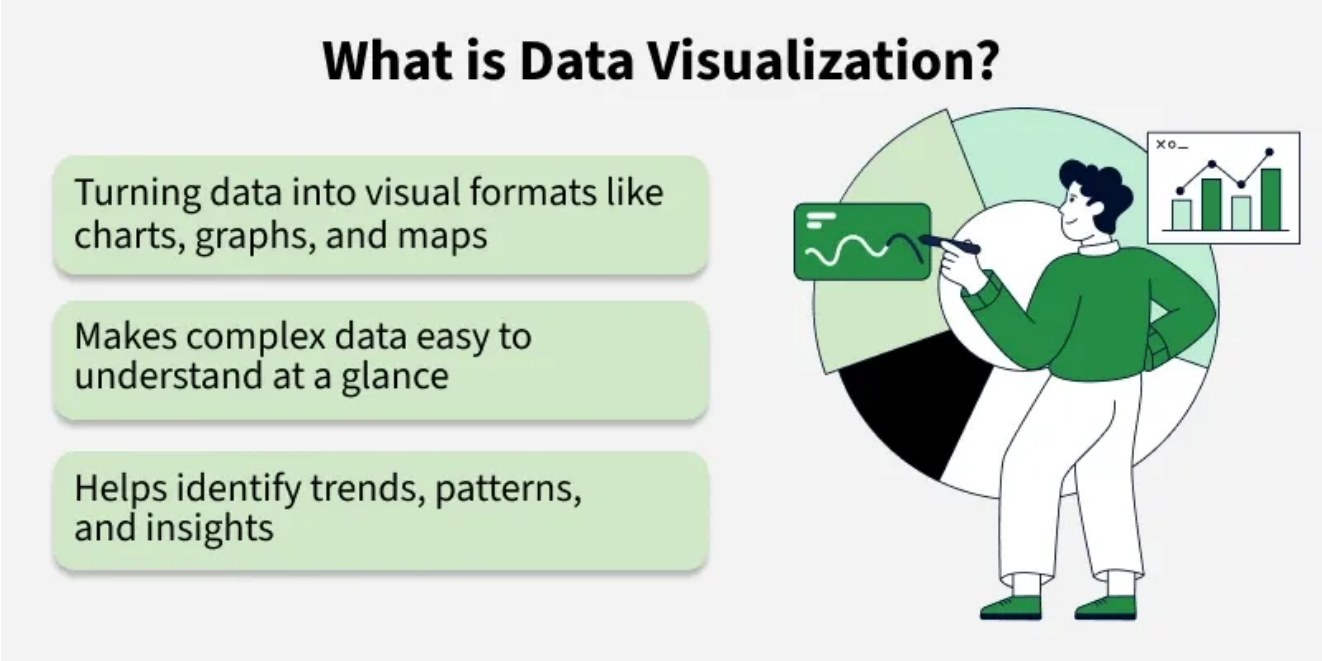
RGUKT IIIT ONG

Contents

- Introduction to Visualization Tools
  - Matplotlib
  - Line Plots
  - Area Plots
  - Histogram
  - Bar chart
  - Pie chart
  - Box Plots
  - Scatter Plots
  - Bubble Plots
- Advanced Visualization Tools
  - Waffle Chart
  - Word Cloud
  - Seaborn and Regression Plots
- Creating Maps & Visualization Geospatial Data
  - Folium Maps with Markers
  - Choropleth maps

Introduction

- Data visualization uses charts, graphs and maps to present information clearly and simply.
- It turns complex data into visuals that are easy to understand. With large amounts of data in every industry, visualization helps spot patterns and trends quickly, leading to faster and smarter decisions.



Common Types of Data Visualization

- There are various types of visualizations where each has a unique purpose in data representation. Here are the most common types:
- **Charts and Graphs:** They are used to visualize data, with charts comparing data points across categories or showing trends over time and graphs analyzing relationships between variables to identify correlations, trends and outliers. Examples: Bar Charts, Line Charts, Pie Charts, Scatter Plots, Histograms, Box Plots.
- **Maps**: They are used to display geographical data which provides spatial context to trends and patterns. Examples: Geographic Maps, Heat Maps
- **Dashboards:** They combine multiple visualizations into a single interface which provides real-time insights and interactive features for users to explore data.

Importance of Data Visualization

Data visualization is essential for understanding and communicating information effectively. Here are some key reasons why it's important:

1.Simplifies Complex Data: It turns large and complicated data into visual formats like charts and graphs, making the information easier to understand.

2.Reveals Patterns and Trends: It helps identify trends, relationships and patterns that are not easily seen in raw data or tables.

3.Saves Time: Visuals allow quicker interpretation of data, helping users spot key information at a glance instead of manually scanning through numbers.

4.Improves Communication: It makes it easier to explain data insights to others, especially those who may not be familiar with the technical details.

5.Tells a Clear Story: Data visuals guide the audience through the information step-by-step, making it easier to reach conclusions and make informed decisions.

Real-World Use Cases for Data Visualization

- Data visualization is used across various industries to improve decision-making and drive results. Here are a few examples:
- **Business Analytics:** Used to monitor company performance, track KPIs and make data-driven decisions by visualizing trends, sales and customer metrics.
- **Healthcare:** Helps in analyzing patient records, tracking disease outbreaks and managing hospital operations through easy-to-read charts and dashboards.
- **Sports:** Used to visualize player statistics, team performance and match outcomes, helping coaches and analysts improve strategies and training plans.
- **Retail and E-commerce:** Enables tracking of sales, customer preferences and inventory levels, helping businesses adjust stock and marketing efforts effectively.

Challenges in Data Visualization

- **Data Quality**: Accuracy of visualizations depends on the quality of the data. If the data is inaccurate or incomplete, the insights from the visualization will be misleading.
- **Over-Simplification**: Simplifying data too much can lead to important details being lost like using a pie chart that oversimplifies complex relationships between categories.
- **Choosing the Right Visualization**: Using the wrong type of visualization can distort the message. For example, a pie chart might not work well with many categories which leads to confusion.
- **Overload of Information**: Too much information in a visualization can overwhelm viewers. It's important to focus on key data points and avoid clutter.

Best Practices for Effective Data Visualization

- To ensure our visualizations are impactful and easy to understand we follow these best practices:
- **Audience-Centric Design**: Tailor visualizations to our audience's knowledge. A technical audience may need detailed graphs while a general audience benefits from simpler charts.
- **Design Clarity and Consistency**: Choose the right chart for our data and keep the design clean with consistent colors, fonts and labels and also avoid clutter to ensure clarity.
- **Provide Context**: Always provide context by including labels, titles and data source acknowledgments. This helps viewers to understand the significance of the data and builds trust in the results.
- **Interactive and Accessible Design**: Make visualizations interactive with features like tooltips and filters and ensure accessibility for all users regardless of device or visual needs.

- Python provides various libraries that come with different features for visualizing data. All these libraries come with different features and can support various types of graphs. In this tutorial, we will be discussing four such libraries.
- Matplotlib
- Seaborn
- Bokeh
- Plotly

Matplotlib

- Matplotlib is an easy-to-use, low-level data visualization library that is built on NumPy arrays. It consists of various plots like scatter plot, line plot, histogram, etc. Matplotlib provides a lot of flexibility.

Seaborn

- **Seaborn** is a high-level interface built on top of the Matplotlib. It provides beautiful design styles and color palettes to make more attractive graphs.
- Seaborn is built on the top of Matplotlib, therefore it can be used with the Matplotlib as well. Using both Matplotlib and Seaborn together is a very simple process. We just have to invoke the Seaborn Plotting function as normal, and then we can use Matplotlib's customization function.

Bokeh

- Bokeh is mainly famous for its interactive charts visualization. Bokeh renders its plots using HTML and JavaScript that uses modern web browsers for presenting elegant, concise construction of novel graphics with high-level interactivity.

Plotly

- Plotly has hover tool capabilities that allow us to detect any outliers or anomalies in numerous data points.
- It allows more customization.
- It makes the graph visually more attractive.

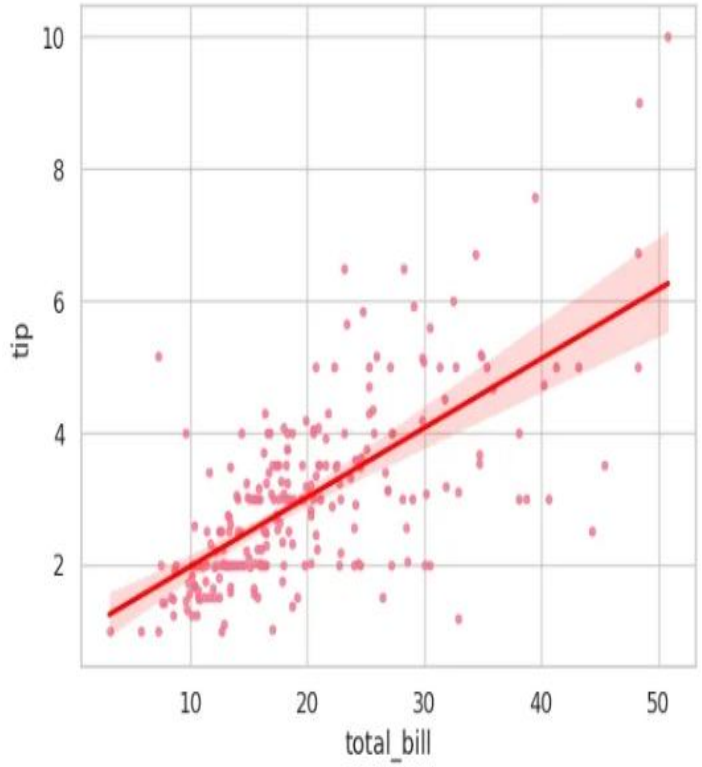
Regression Plots: Visualizing Linear Relationships

- Seaborn simplifies the process of performing and visualizing regressions specifically linear regressions which is important for identifying relationships between variables, detecting trends and making predictions. It supports two primary functions for regression visualization:
- regplot(): This function plots a scatter plot along with a linear regression model fit.
- lmplot(): This function also plots linear models but provides more flexibility in handling multiple facets and datasets.

Example: Let's use a simple dataset to visualize a linear regression between two variables: x (independent variable) and y (dependent variable).

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset('tips')

sns.regplot(x='total_bill', y='tip', data=tips, scatter_kws={'s':10}, line_kws={'color':'red'})
plt.show()
```

Output:



Regression Plots

Types of Data Visualization Charts

Basic Charts for Data Visualization

- Basic charts are best suited for displaying simple comparisons, trends over time and basic relationships within the data. These charts are easy to understand and ideal for communicating insights to a broad audience

Types of Basic Charts:

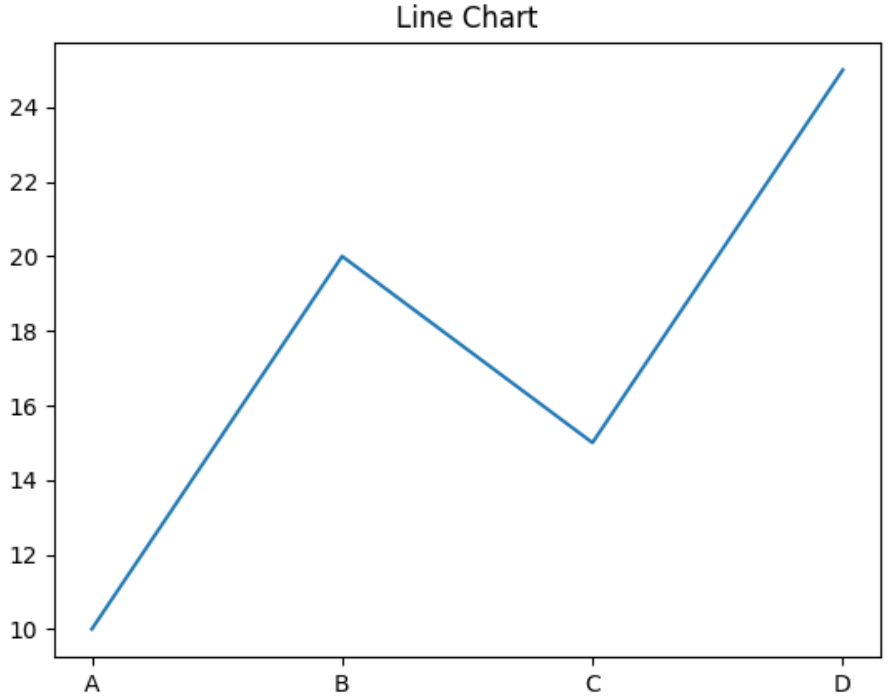
- Line Plots
- Area Plots
- Histograms
- Bar Charts
- Pie Charts
- Box Plots
- Scatter Plots
- Bubble Plots

Line Charts

Line charts show how values change over time by connecting data points with lines. They help visualize trends like increases, decreases or stability.

When to Use:

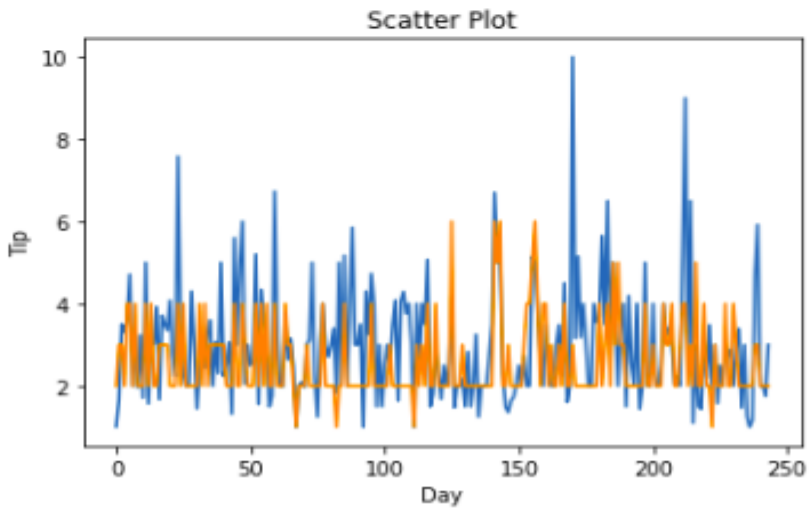
- To track changes over time
- To compare trends across multiple data series
- For time series analysis



Example:

```
import pandas as pd
import matplotlib.pyplot as plt


# reading the database
data = pd.read_csv("tips.csv")

# Scatter plot with day against tip
plt.plot(data['tip'])
plt.plot(data['size'])

# Adding Title to the Plot
plt.title("Scatter Plot")

# Setting the X and Y labels
plt.xlabel('Day')
plt.ylabel('Tip')

plt.show()
```

Output:

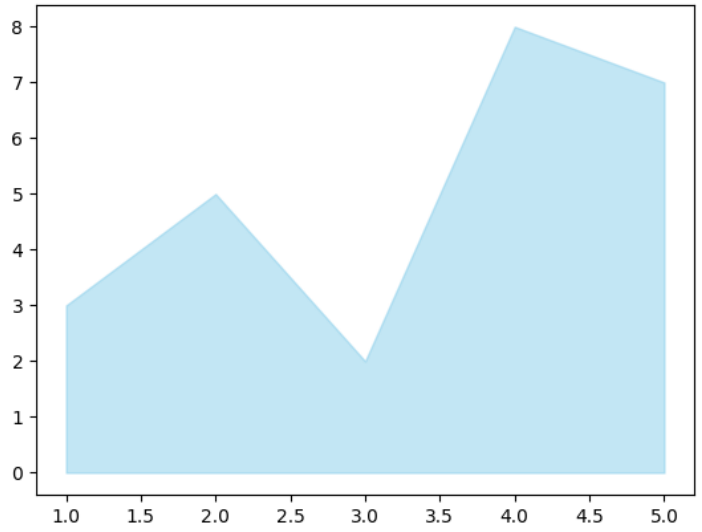


Area Chart

- An area chart shows trends over time by filling the space beneath a line. It's ideal for visualizing time-series data and highlighting changes across periods.

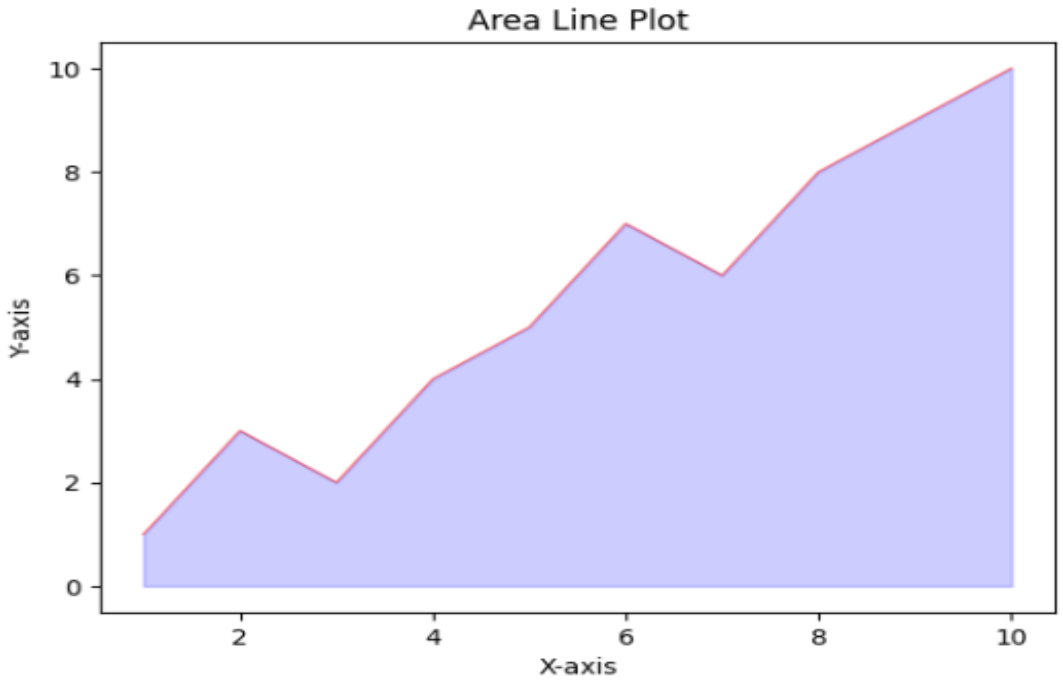
When to Use:

- To track changes or trends over time
- To compare multiple data series
- To highlight patterns like seasonality or cycles



```python
import pandas as pd
import matplotlib.pyplot as plt

# Sample data
df = pd.DataFrame({
    'x': list(range(1, 11)),
    'y': [1, 3, 2, 4, 5, 7, 6, 8, 9, 10]
})

# Create the area line plot
plt.fill_between(df['x'], df['y'], color='blue', alpha=0.2)
plt.plot(df['x'], df['y'], color='red', alpha=0.5, linewidth=0.9)

plt.title("Area Line Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

Output:



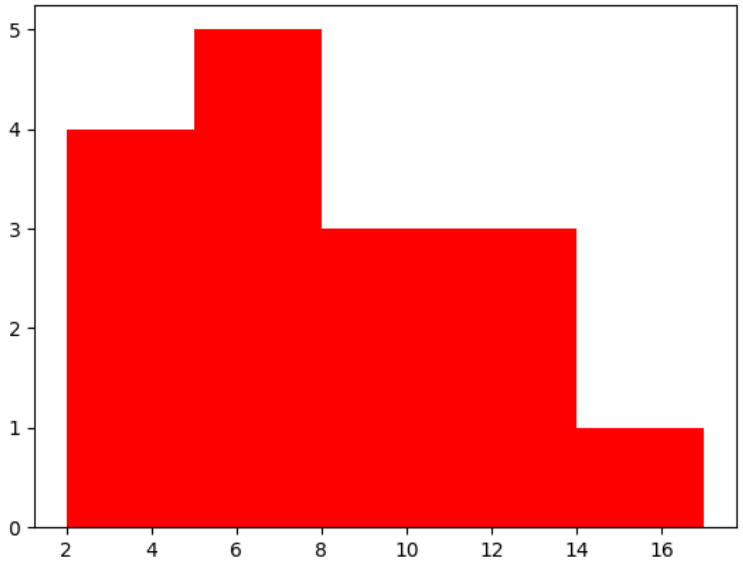
Area line Plot

Histogram

- A histogram displays the distribution of numerical data by grouping values into intervals (bins) and showing their frequency as bars. It helps reveal the shape, spread and patterns in the data.

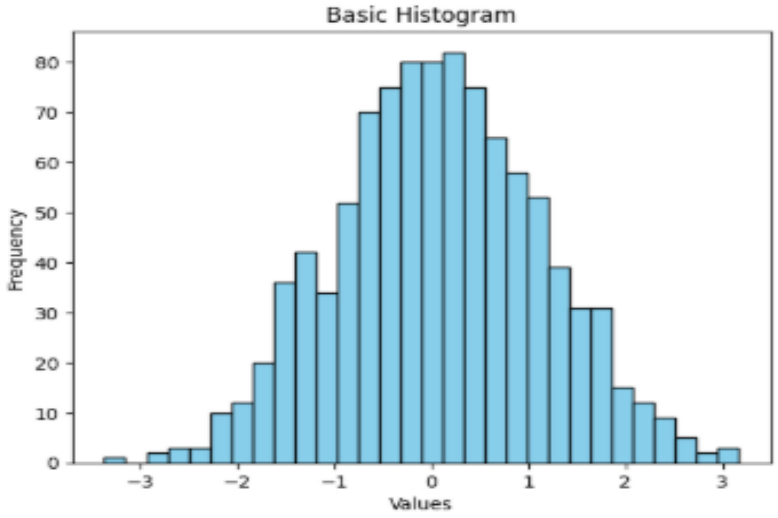
When to Use:

- To visualize the distribution of numerical data
- To explore patterns, trends and outliers



```python
import matplotlib.pyplot as plt
import numpy as np

# Generate random data for the histogram
data = np.random.randn(1000)

# Plotting a basic histogram
plt.hist(data, bins=30, color='skyblue', edgecolor='black')

# Adding labels and title
plt.xlabel('Values')
plt.ylabel('Frequency')
plt.title('Basic Histogram')

# Display the plot
plt.show()
```

Output:

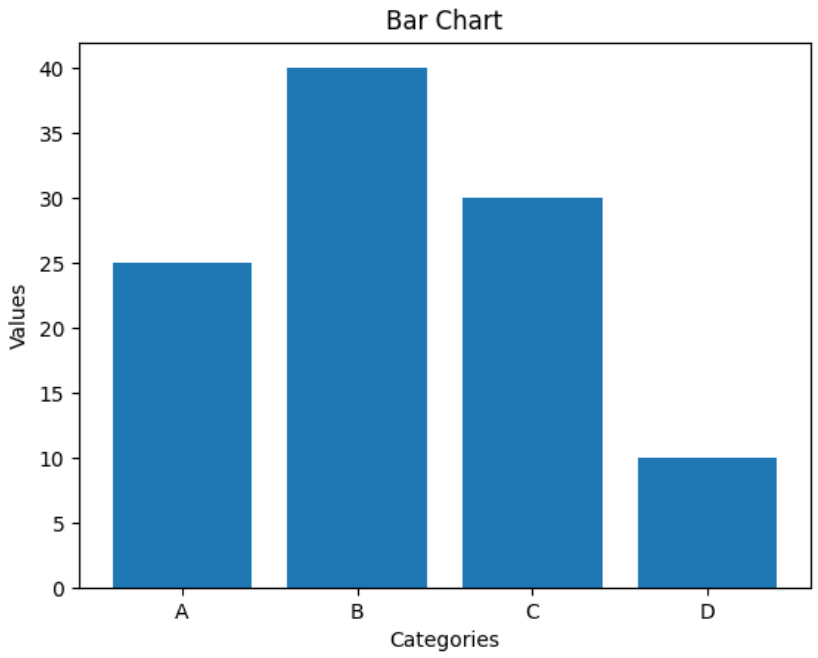


Bar Charts

- Bar charts are used to compare values across different categories using rectangular bars. X-axis shows categories while Y-axis represents values. Common types include horizontal, stacked and grouped bar charts.

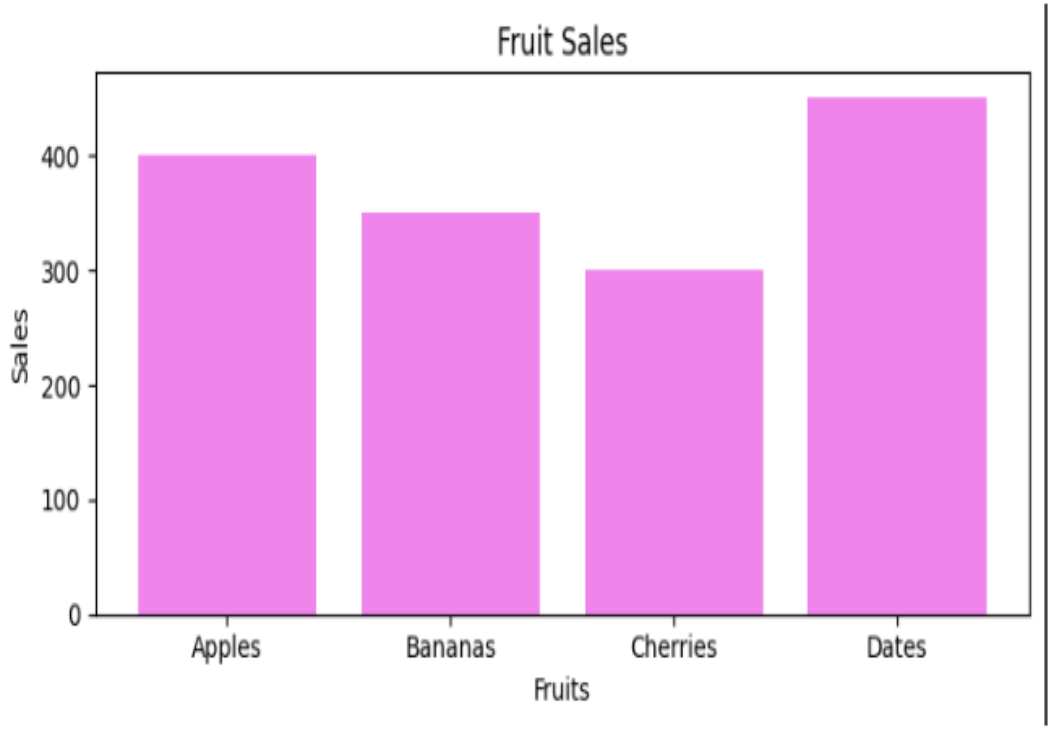
When to Use:

- To compare different categories
- To rank values from highest to lowest
- To show relationships between multiple variables



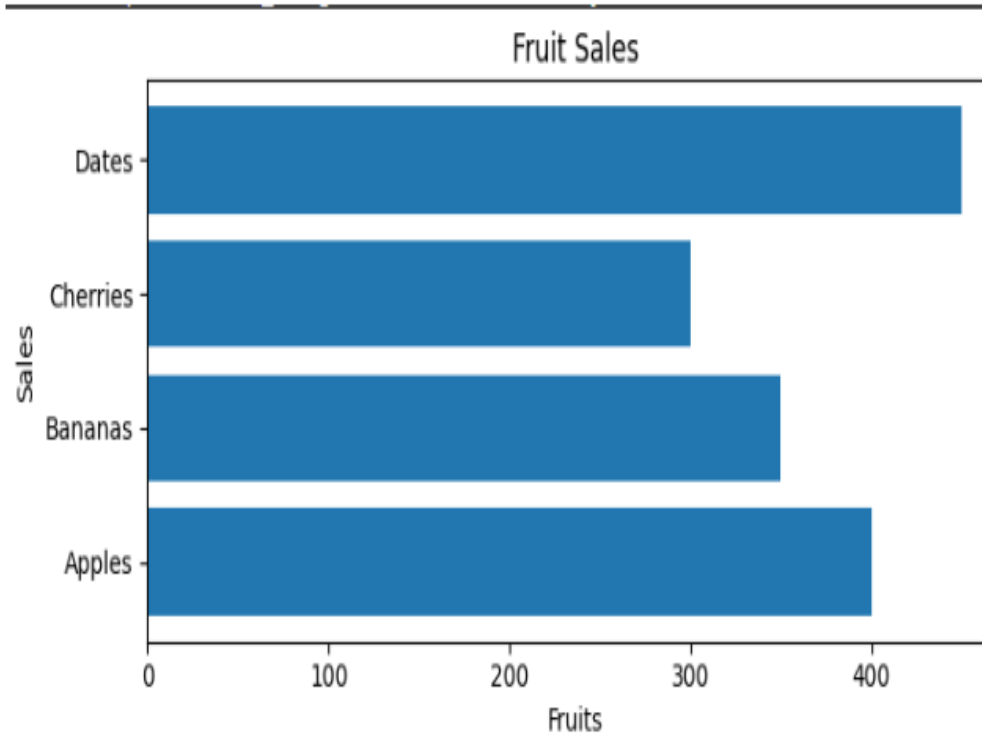
```python
import matplotlib.pyplot as plt
import numpy as np

fruits = ['Apples', 'Bananas', 'Cherries', 'Dates']
sales = [400, 350, 300, 450]

plt.bar(fruits, sales, color='violet')
plt.title('Fruit Sales')
plt.xlabel('Fruits')
plt.ylabel('Sales')
plt.show()
```

Output:




```python
import numpy as np

fruits = ['Apples', 'Bananas', 'Cherries', 'Dates']
sales = [400, 350, 300, 450]

plt.barh(fruits, sales)

plt.title('Fruit Sales')
plt.xlabel('Fruits')
plt.ylabel('Sales')
plt.show()
```

Output:

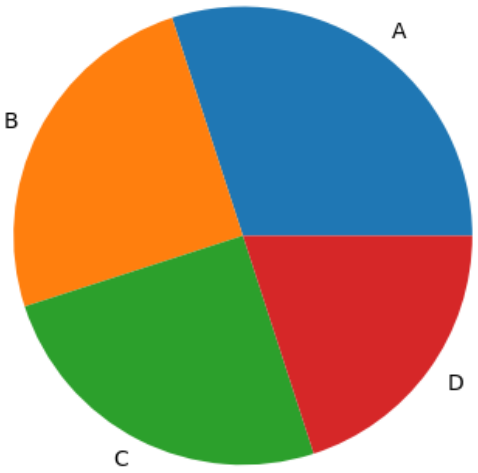


Pie Charts

- Pie charts are round charts divided into slices, where each slice shows a part of the whole. The size of each slice represents its percentage.

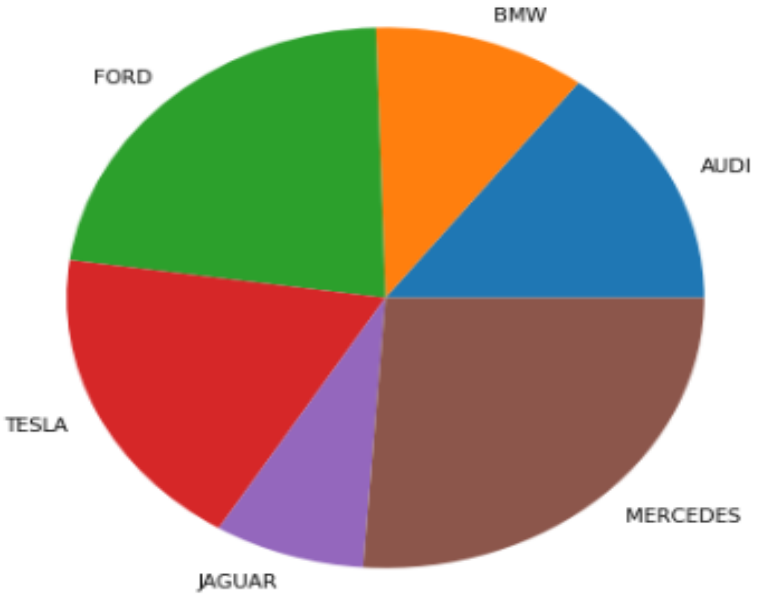
When to Use:

- To show how different parts contribute to a whole
- To highlight a dominant category



```python
# Import libraries
from matplotlib import pyplot as plt
import numpy as np


# Creating dataset
cars = ['AUDI', 'BMW', 'FORD',
        'TESLA', 'JAGUAR', 'MERCEDES']

data = [23, 17, 35, 29, 12, 41]

# Creating plot
fig = plt.figure(figsize=(10, 7))
plt.pie(data, labels=cars)

# show plot
plt.show()
```

Output:

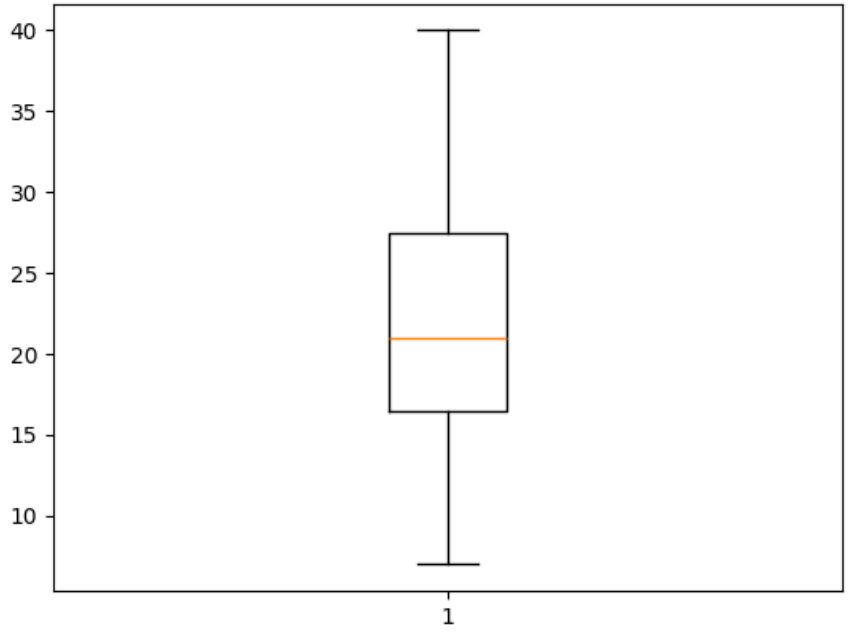


Box Plot (Box-and-Whisker Plot)

- A box plot displays distribution of numerical data, showing median, quartiles and outliers. It's useful for understanding variability and detecting unusual values.

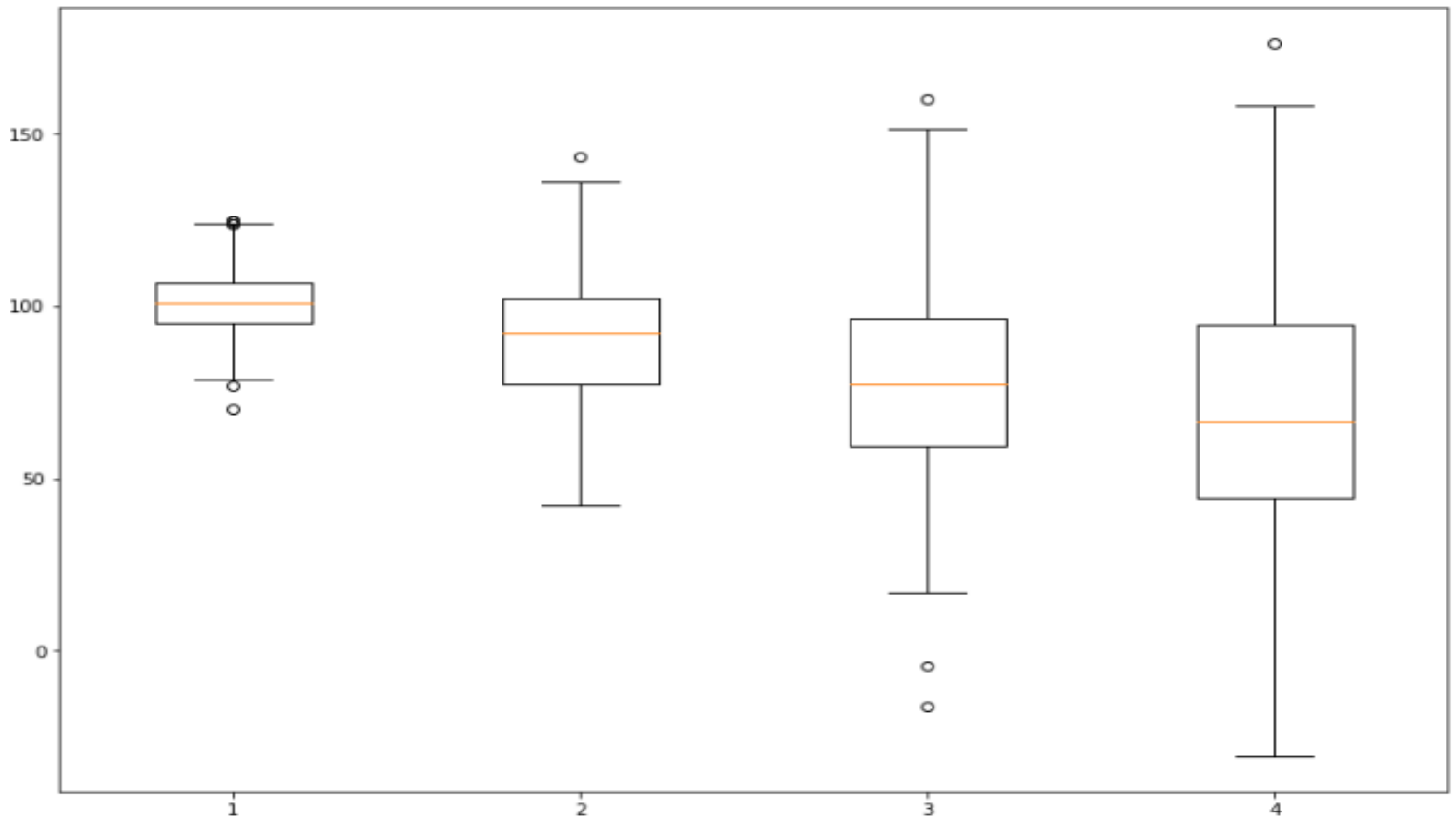
When to Use:

- To detect outliers in data
- To compare distributions across groups
- To visualize spread and variability



```python
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(10)

d_1 = np.random.normal(100, 10, 200)
d_2 = np.random.normal(90, 20, 200)
d_3 = np.random.normal(80, 30, 200)
d_4 = np.random.normal(70, 40, 200)
d = [d_1, d_2, d_3, d_4]

fig = plt.figure(figsize =(10, 7))
ax = fig.add_axes([0, 0, 1, 1])
bp = ax.boxplot(d)

plt.show()
```

Output:

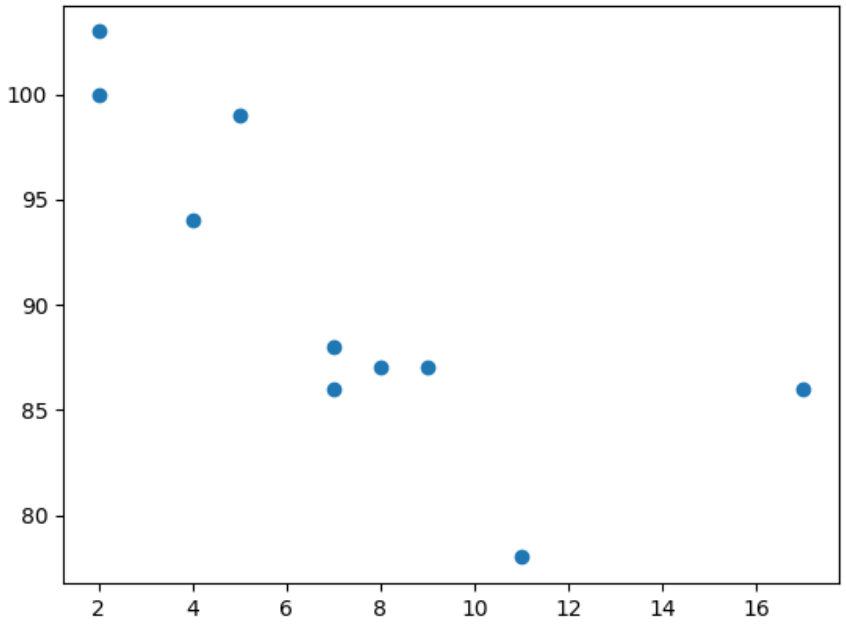


Scatter Chart (Plots)

- Scatter charts use dots to show relationship between two numerical variables. X-axis shows the independent variable and Y-axis shows the dependent variable.

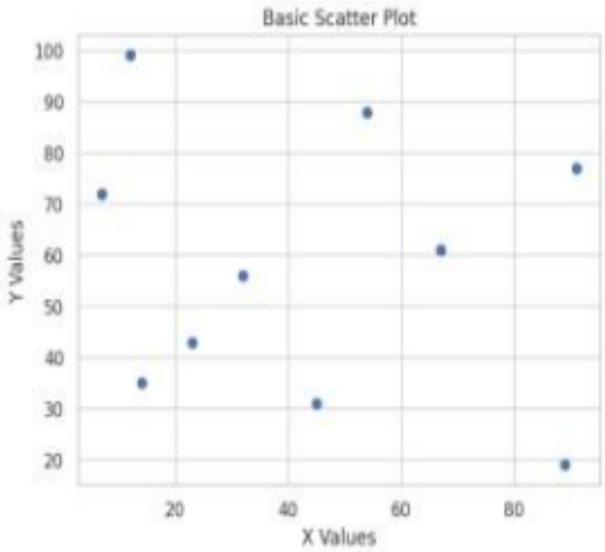
When to Use:

- To observe relationships between two variables
- To detect patterns, clusters or outliers in data



```python
import matplotlib.pyplot as plt
import numpy as np

x = np.array([12, 45, 7, 32, 89, 54, 23, 67, 14, 91])
y = np.array([99, 31, 72, 56, 19, 88, 43, 61, 35, 77])

plt.scatter(x, y)
plt.title("Basic Scatter Plot")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.show()
```

Output



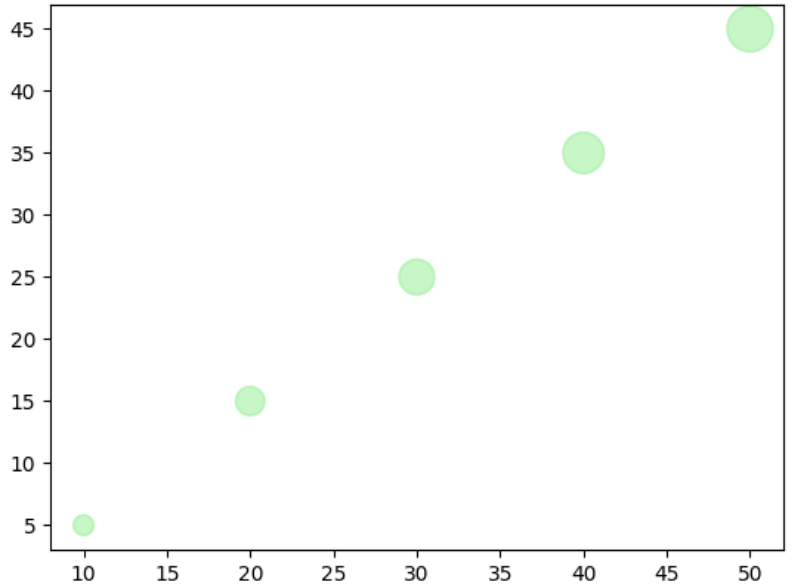
Using matplotlib.pyplot.scatter()

Bubble Chart

- A bubble chart displays data points as circles, where size and color of each bubble represent additional variables. It's useful for visualizing three or more dimensions in a single chart.

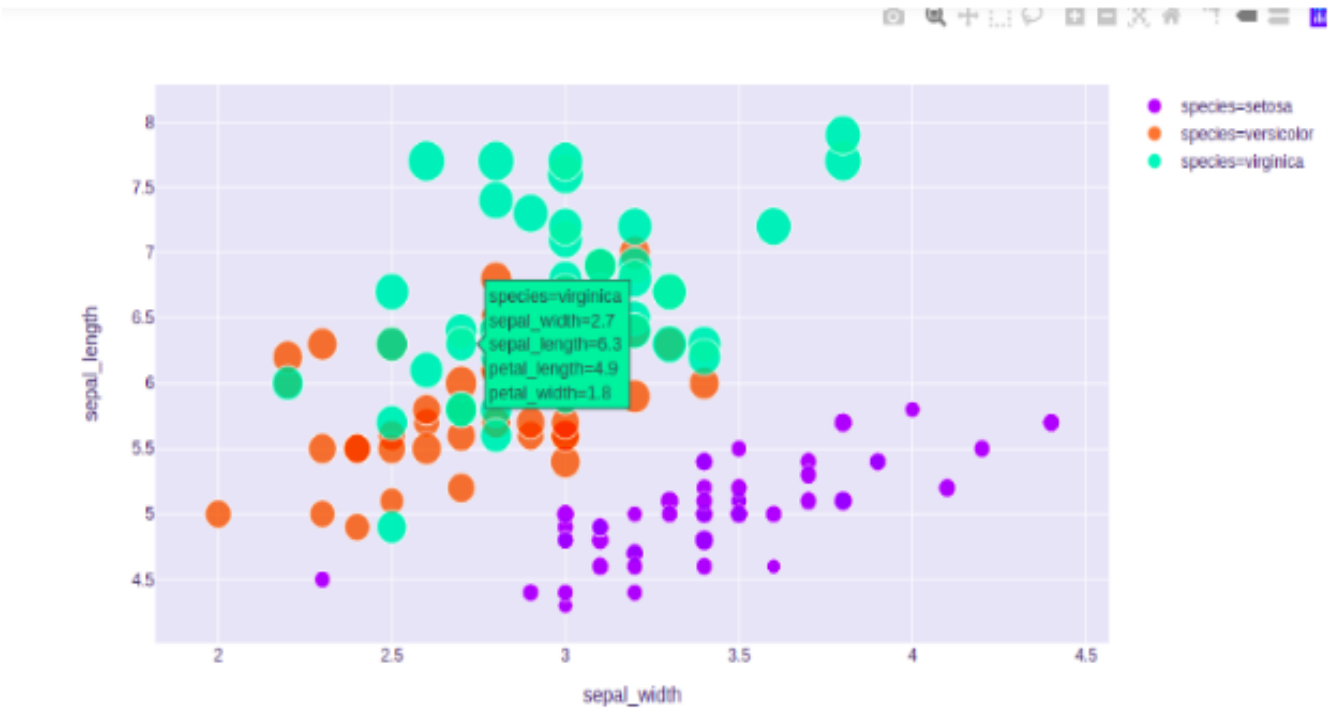
When to Use:

- To compare multiple variables at once
- To represent data using size and color
- To visualize relationships and detect patterns



```python
import plotly.express as px

df = px.data.iris()

fig = px.scatter(df, x="sepal_width", y="sepal_length",
                 color="species",
                 size='petal_length',
                 hover_data=['petal_width'])

fig.show()
```

Output:



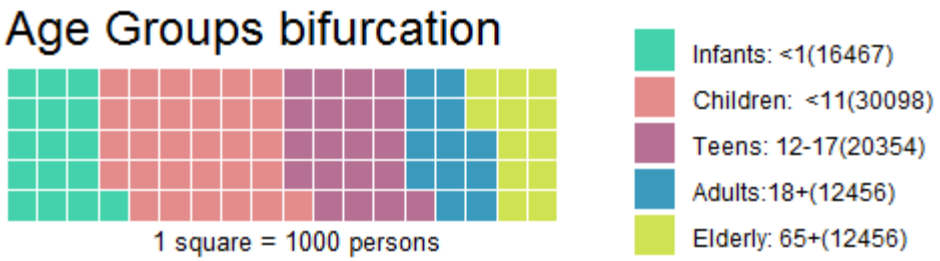
Advanced Charts for Data Visualization

Advanced charts are designed to handle more complex data. They help analyze multiple variables, uncover deeper insights and reveal patterns that might be missed with basic visuals.

- Waffle Charts
- Word Clouds
- **Choropleth Map**
- Heat Map
- Parllel Coordinates
- Tree Map
- Sankey Diagram
- Radar Chart
- Network Graph
- Donut Chart
- Gauge Chart
- Sunburst Chart
- Hexbin Plot
- Violin Plot
- Pictogram Chart
- Streamgraph
- Bullet Graph
- Gantt Chart
- Waterfall Chart

Waffle chart

- A waffle chart is a square grid chart that fills up to a certain percentage. It is used to visualize different types of data in a very simple yet effective way. One similar example of a waffle chart could be the Battery symbols on our modern-day smartphones. A waffle chart is a wonderful method of representation of numeric data, and it can be used as a replacement for a Pie Chart.



```python
# python program to generate Waffle Chart

# importing all necessary requirements
import pandas as pd
import matplotlib.pyplot as plt
from pywaffle import Waffle

# creation of a dataframe
data ={'phone': ['Xiaomi', 'Samsung',
                 'Apple', 'Nokia', 'Realme'],
       'stock': [44, 12, 8, 5, 3]
     }

df = pd.DataFrame(data)

# To plot the waffle Chart
fig = plt.figure(
    FigureClass = Waffle,
    rows = 5,
    values = df.stock,
    labels = list(df.phone)
)
```

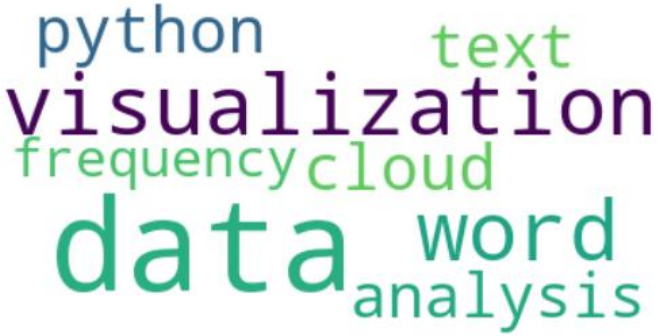
Output:

Word Cloud

- A word cloud visually displays words sized by their frequency in a text, common words appear larger. It helps quickly identify key terms or themes in text data.

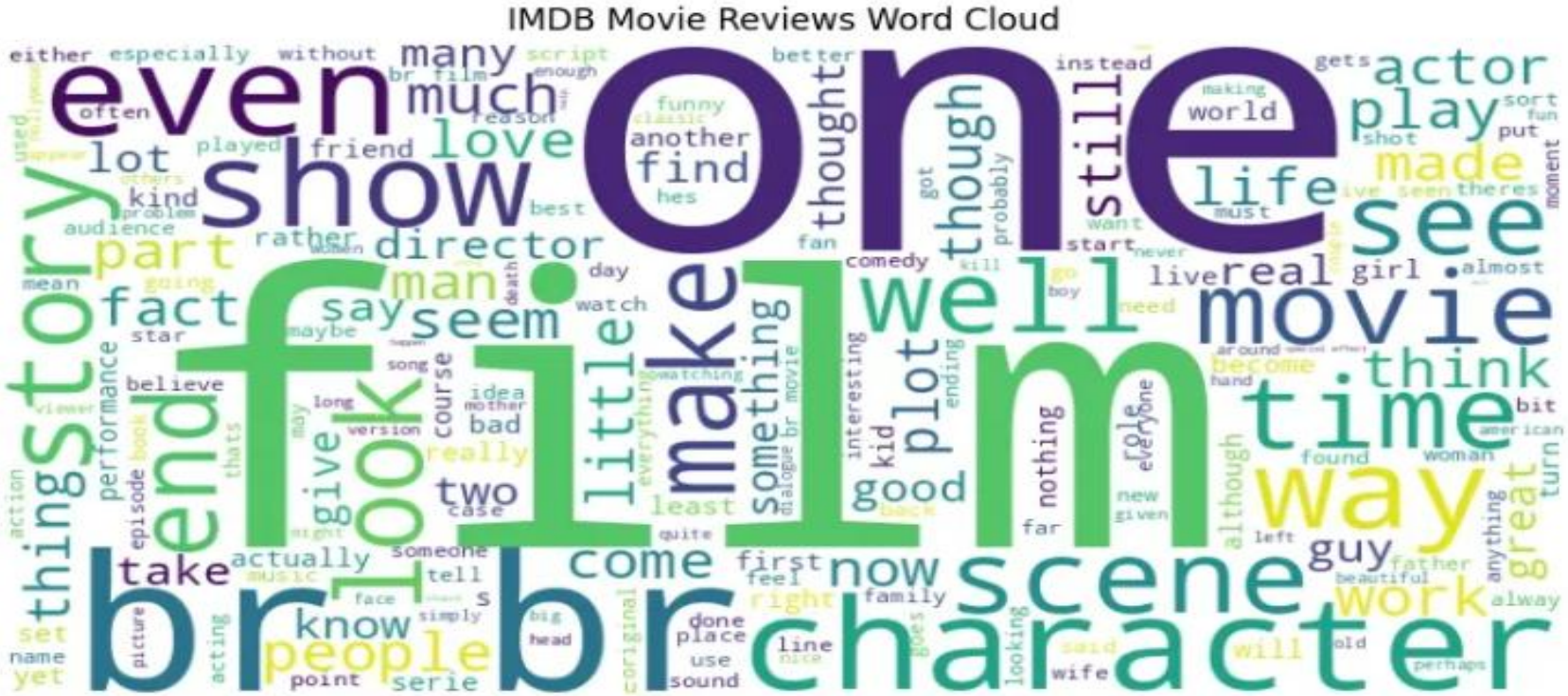
When to use:

- To highlight key themes or topics in large text data.
- To visualize word frequency in text analysis.
- To emphasize important terms in qualitative or sentiment analysis



```python
from wordcloud import WordCloud
import matplotlib.pyplot as plt

wordcloud = WordCloud(width=800, height=400, background_color='white').generate(text)

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title("IMDB Movie Reviews Word Cloud")
plt.show()
```

Output:

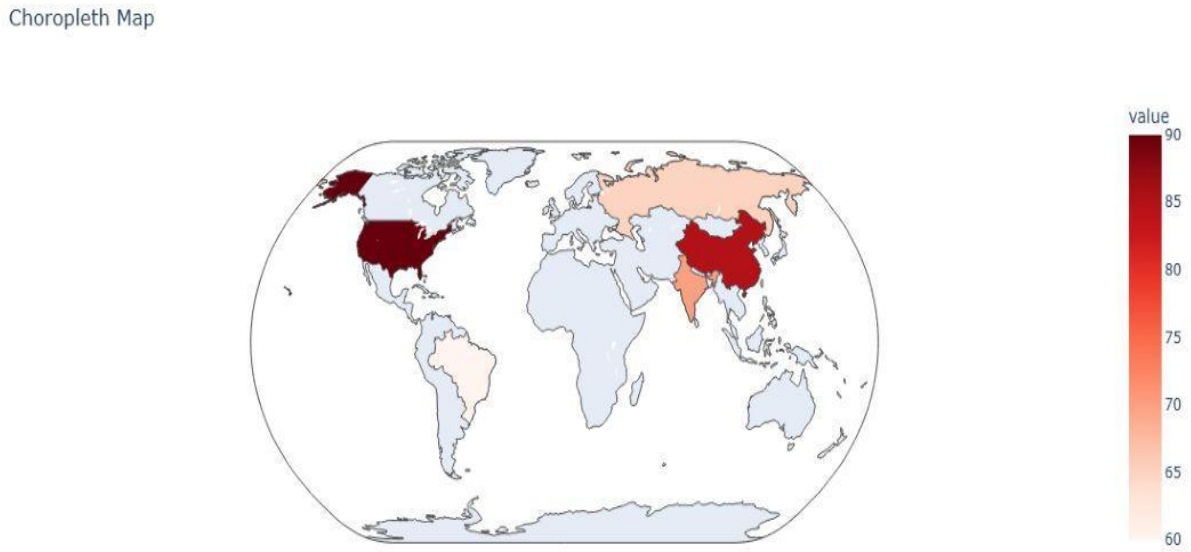

IMDB Movie Reviews Word Cloud
one
film
br
br
character
even
show
story
movie
time
way
scene
well
make
end
look
people
actor
play
see
made
life
still
though
thought
love
much
many
director
fact
part
seem
little
something
plot
good
think
real
work
guy
come
now
take
two
give
know
actually
another
find
friend
lot
kind
audience
rather
say
man
watch
comedy
live
girl
almost
great
nothing
role
everyone
although
found
woman
turn
anything
father
beautiful
always
will
wife
line
place
use
sound
done
head
point
serie
name
yet
set
picture
acting
original
looking
family
feel
first
right
tell
someone
music
face
simply
big
bad
really
course
idea
mother
version
case
least
quite
far
new
given
left
interesting
ending
watching
everything
dialogue
br movie
kid
comedy
become
hand
around
boy
need
start
never
want
must
probably
fan
death
day
maybe
star
song
long
may
thats
viewer
book
episode
action
performance
believe
mean
going
problem
used
others
hes
best
reason
played
often
either
especially
without
enough
br film
script
funny
classic
better
instead
gets
making
world
shot
put
sort
fun
moment
theres
ive
seen
must
almost
girl
boy
need
hand
bit
american
anything
turn
great
always
beautiful
father
old
perhaps
said
wife
will
left
found
woman
although
role
everyone
nothing
far
new
given

Choropleth Map

- A choropleth map uses color shading to represent data across geographic areas. It's useful for showing differences like population density, income levels or disease spread across regions.

When to Use:

- To compare data across locations
- To spot geographic patterns and trends



Folium

- One of the most important tasks for someone working on datasets with countries, cities, etc. is to understand the relationships between their data's physical location and their geographical context. And one such way to visualize the data is using **Folium**.
- Folium is a powerful data visualization library in Python that was built primarily to help people visualize geospatial data. With Folium, one can create a map of any location in the world. Folium is actually a python wrapper for leaflet.js which is a javascript library for plotting interactive maps.
- ***Syntax:*** *folium.Map(location,tiles= "OpenStreetMap" zoom_start=4)*
- ***Parameters:***
- ***location****: list of location coordinates*
- ***tiles****: default is OpenStreetMap. Other options: tamen Terrain, Stamen Toner, Mapbox Bright etc.*
- ***zoom_start****: int*

import the folium, pandas libraries

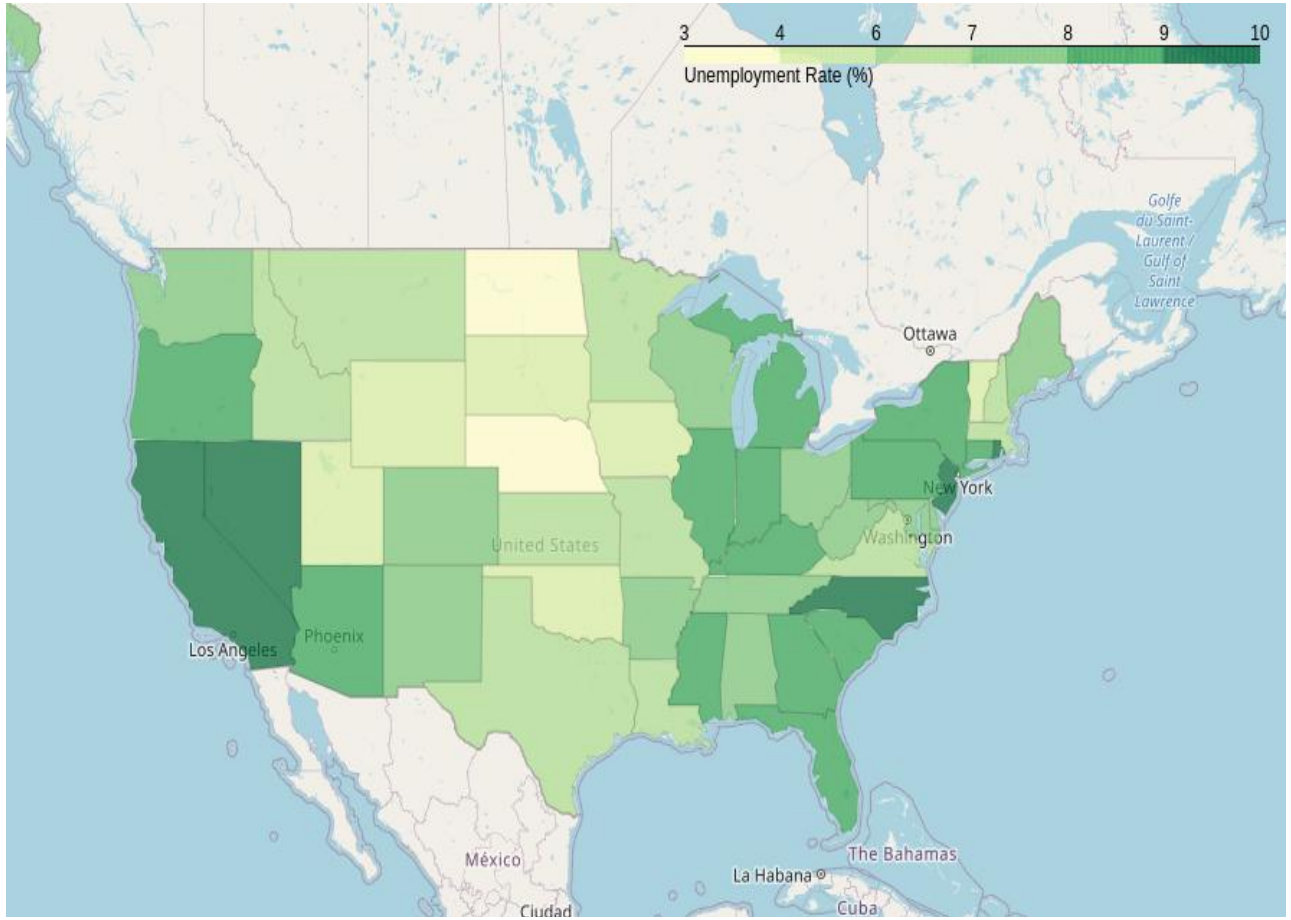
import folium

- import pandas as pd
- # initialize the map and store it in a m object
- m = folium.Map(location = [40, -95],
- zoom_start = 4)

- # show the map
- m.save('my_map.html')

- ***Syntax:*** *folium.Choropleth(geo_data,name,data,columns,fill_color, fill_opacity, line_opacity, key_on,legend_name)*
- ***Parameters:***
- *geo_data: a set of geographic regions and their boundary coordinates*
- *name: String (name of our map)*
- *data: a numeric value for each region, used for the color*
- *columns: list (columns we need to work on)*
- *fill_color: Color of the map, eg: YlGn*
- *fill_opacity: opacity of the colors filled*
- *line_opacity: opacity of the border lines*
- *legend_name: String*

```
folium.Choropleth(
    # geographical locations
    geo_data = state_geo,
    name = "choropleth",
    # the data set we are using
    data = state_data,
    columns = ["State", "Unemployment"],
    # YlGn refers to yellow and green
    fill_color = "YlGn",
    fill_opacity = 0.7,
    line_opacity = .1,
    key_on = "feature.id",
    legend_name = "Unemployment Rate (%)",
).add_to(m)
m.save('final_map.html')
```

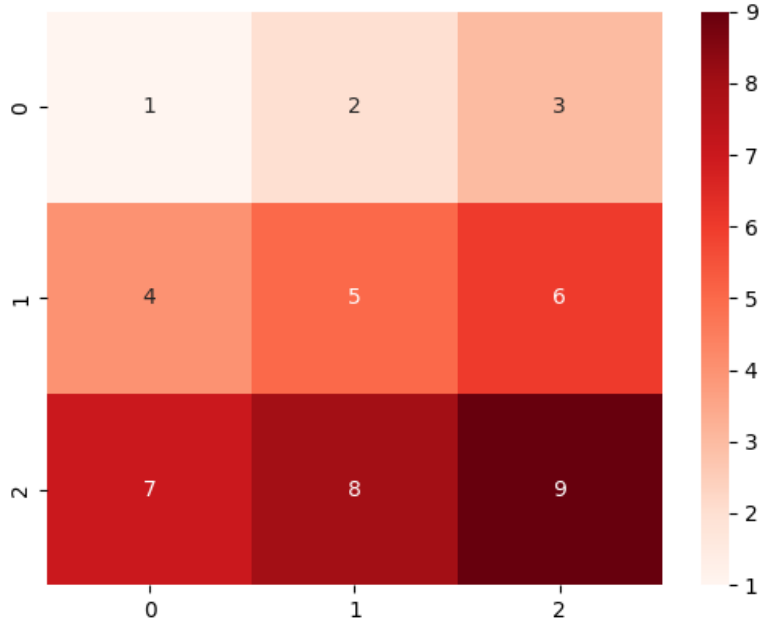


Heatmap

- A heatmap displays data in a matrix format using color to represent values. It's ideal for spotting patterns, correlations and variations in large datasets.

When to Use:

- To identify clusters or groupings in data
- To visualize correlations between variables
- For risk analysis in fields like finance or network security

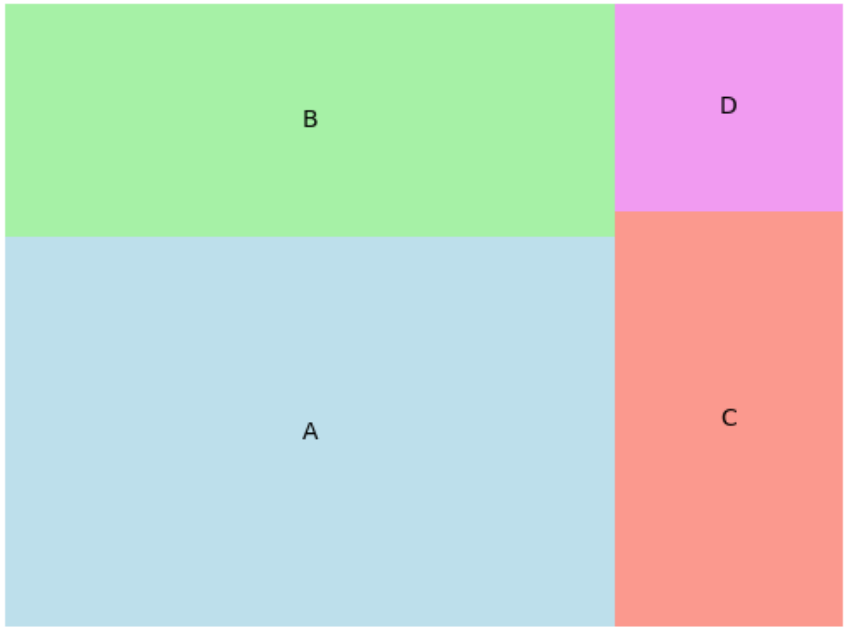


Tree Map

- A tree map visualizes hierarchical data using nested rectangles, where the size of each represents a value. It’s useful for showing structure and comparing proportions within a hierarchy.

When to Use:

- To display hierarchical data
- To compare proportions within levels
- To visualize large datasets compactly

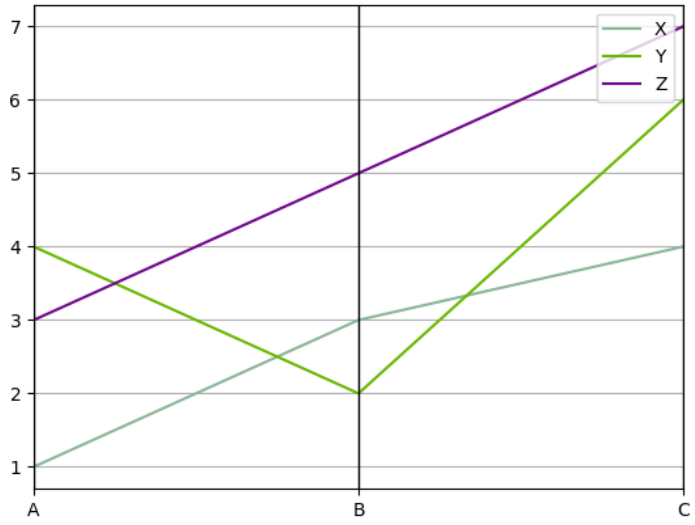


Parallel Coordinates

- Parallel coordinates display multivariate data using lines that connect values across multiple axes. Each line represents a data point across several variables.

When to Use:

- To analyze multiple variables at once
- To visualize relationships or clusters
- To detect outliers in the data

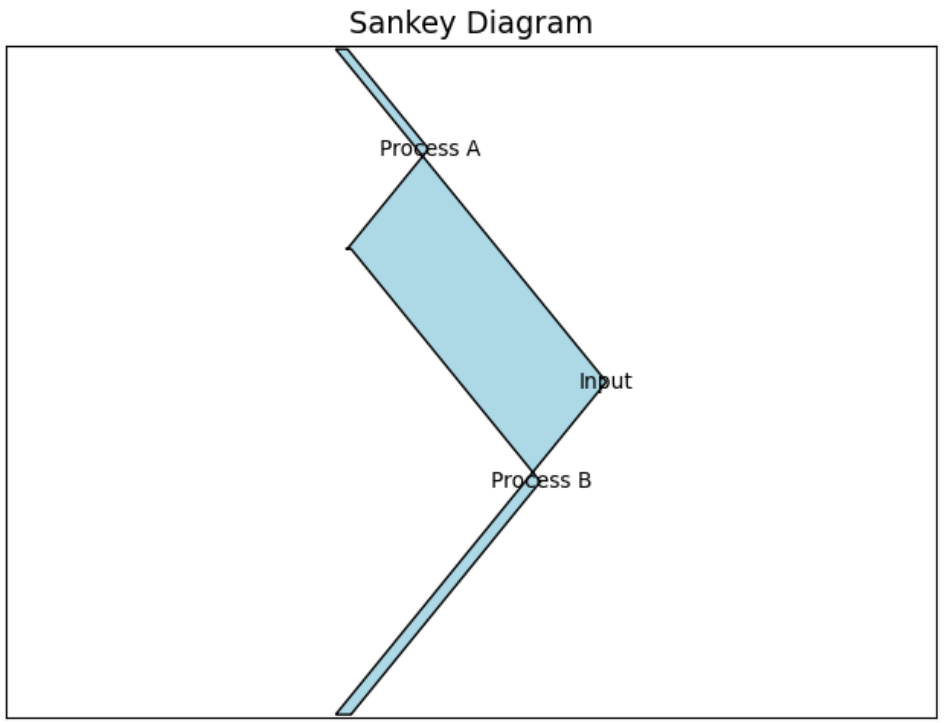


Sankey Diagram

- A Sankey diagram shows the flow of data or resources between points (nodes) using arrows, where the width of each arrow represents the amount of flow. It's great for visualizing complex systems and spotting inefficiencies.

When to Use:

- Visualize Flows: Understand how resources move through a process.
- Find Bottlenecks: Identify points where flow slows or drops.
- Compare Scenarios: Track flow changes across time or conditions.

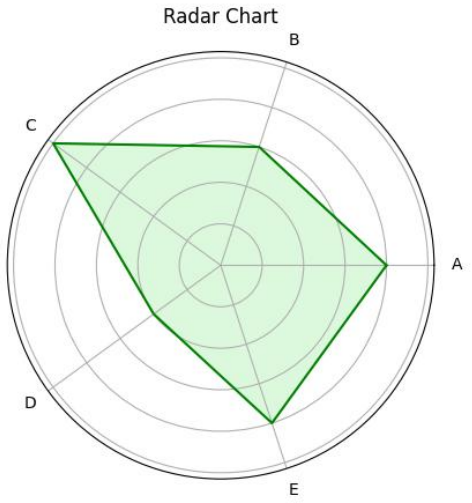


Radar Chart (Spider Chart)

- A radar chart displays multivariate data using axes from a central point ideal for comparing variables across categories. Common in performance analysis, sports and decision-making.

When to Use:

- Multi-Criteria Comparison: Compare multiple variables at once.
- Strengths & Weaknesses: Visualize strong and weak areas.
- Pattern Recognition: Spot similarities or differences between categories.

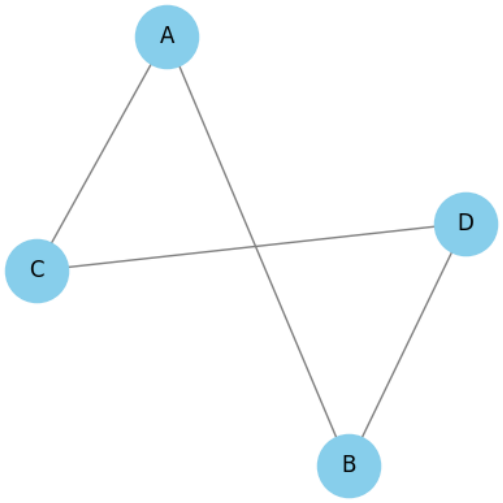


Network Graph

- A network graph shows relationships between entities as nodes and links (edges). It helps visualize complex networks like social media, transport routes or biological systems.

When to Use:

- Relationship Visualization: Helps show connections or interactions between entities.
- Community Detection: Useful to identify clusters or groups within a network.
- Path Analysis: Shows shortest or most efficient routes between nodes.

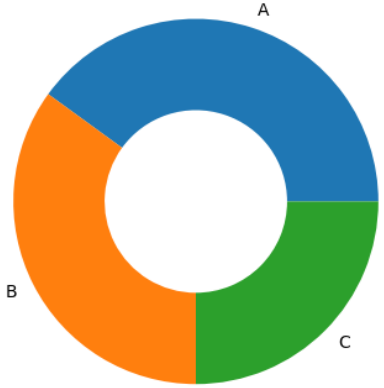


Donut or Doughnut chart

- A donut chart is a circular chart like a pie chart but with a hole in the center. Each slice shows a category's contribution to the whole, making it visually cleaner and ideal for comparisons.

When to Use:

- Show part-to-whole relationships like sales or market share.
- Track progress toward a goal (e.g., 70% completed).
- Best when comparing a few categories.

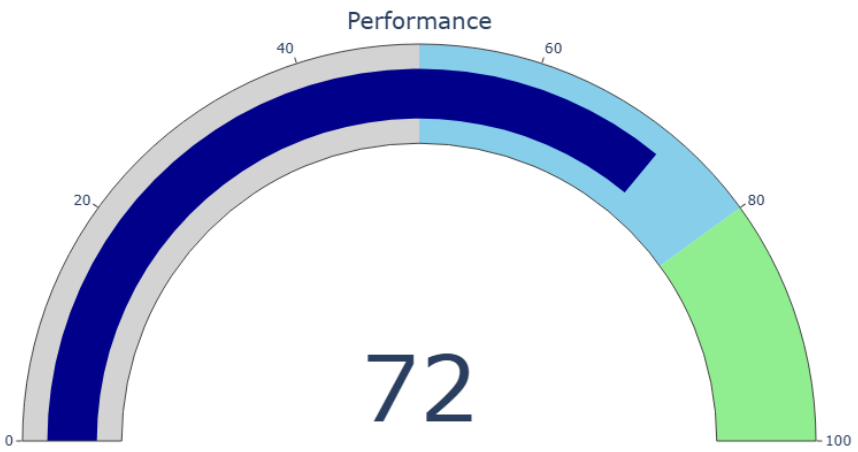


Gauge Chart

- A Gauge chart shows progress toward a goal using a dial-like arc, similar to a speedometer. It is useful for tracking a single value like a KPI or project status.

When to Use:

- To monitor key metrics (e.g., sales, satisfaction) toward targets
- In KPI dashboards to track progress
- For project status tracking against deadlines



Sunburst Chart

- A sunburst chart shows hierarchical data as nested rings. Each ring represents a level in the hierarchy making it useful for visualizing multi-level data structures like categories, subcategories or organizational hierarchies.

When to Use:

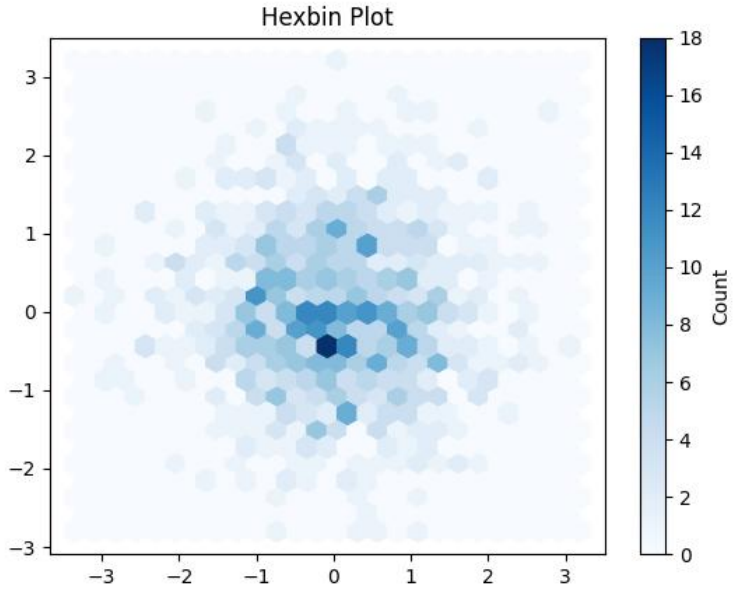
- To visualize hierarchical structures like org charts or file systems.
- To explore proportions and relationships within nested datasets.
- To simplify complex data in a clean, interactive layout.

Hexbin Plot

- A hexbin plot groups points into hexagonal bins and colors them by how many points fall in each bin. It's useful for large datasets to show density clearly without overplotting.

When to Use:

- To show data density in a 2D space.
- To spot clusters or patterns in large scatter data.
- To manage overlapping points in big datasets visually.

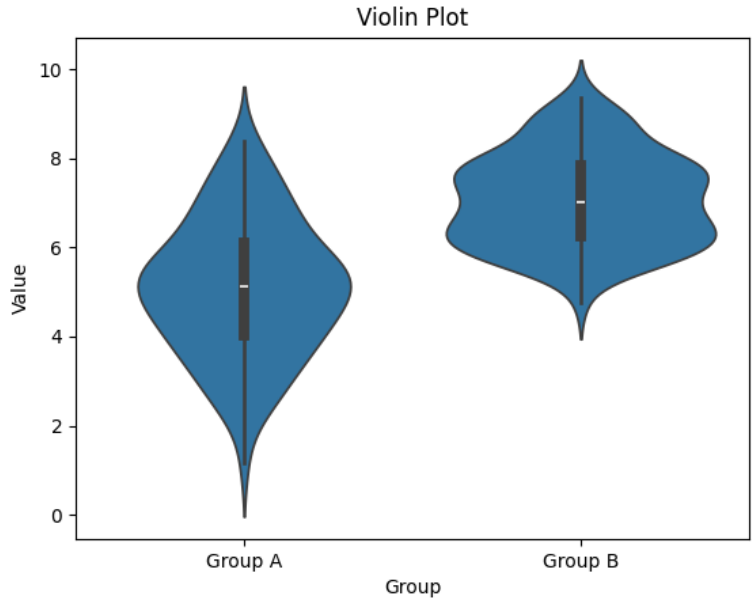


Violin Plot

- A violin plot combines a box plot and a density plot to show distribution and summary statistics of data. It helps visualize the shape, spread and center of the data across categories.

When to Use:

- Compare distribution of continuous data across groups.
- Visualize data shape, spread, skewness or multimodal patterns.
- Highlight detailed distribution and outliers in a clean layout.

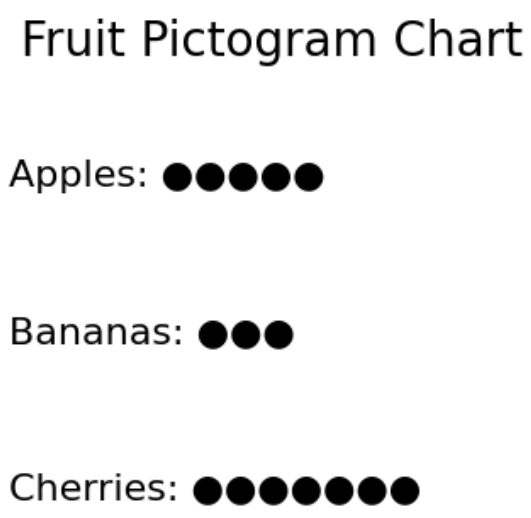


Pictogram Chart

- A pictogram chart uses icons or symbols to visually represent data. The number or size of icons reflects the value. It's helpful for showing qualitative or categorical data in a simple way.

When to use:

- To present data using symbols, especially for non-numeric info.
- To communicate with audiences of all literacy levels.
- To highlight key trends using clear, recognizable icons.

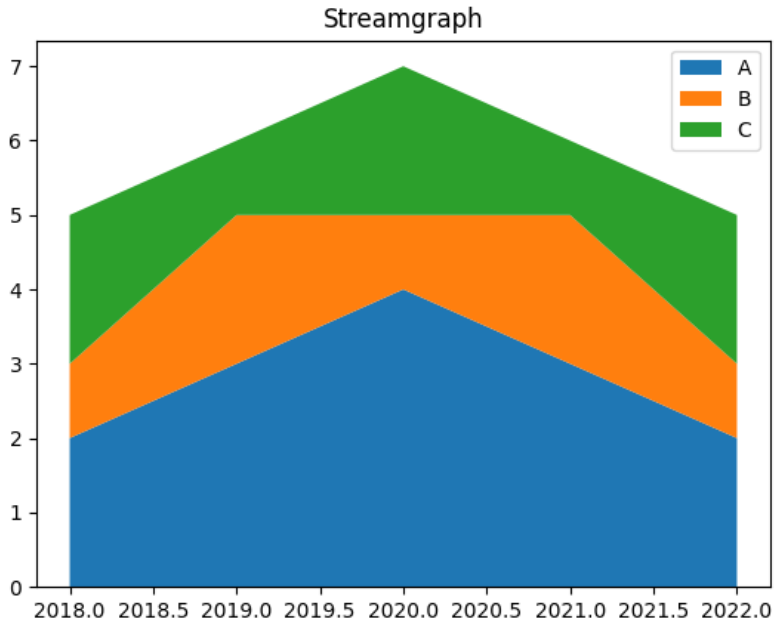


Streamgraph

- A streamgraph displays changes in data over time using flowing, stacked areas. It shows how different categories contribute to the whole across time.

When to Use:

- To show trends and changes in category distribution over time
- To compare the size of different groups over time
- To highlight patterns in a smooth and visually appealing way

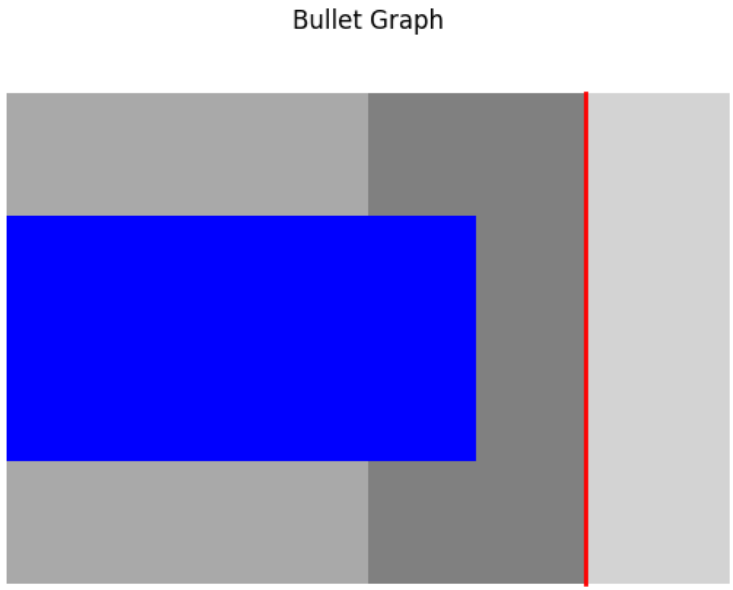


Bullet Graph

- A bullet graph is a variant of a bar chart but it includes markers and reference lines to show progress toward a goal. It's useful for tracking performance against a target.

When to use:

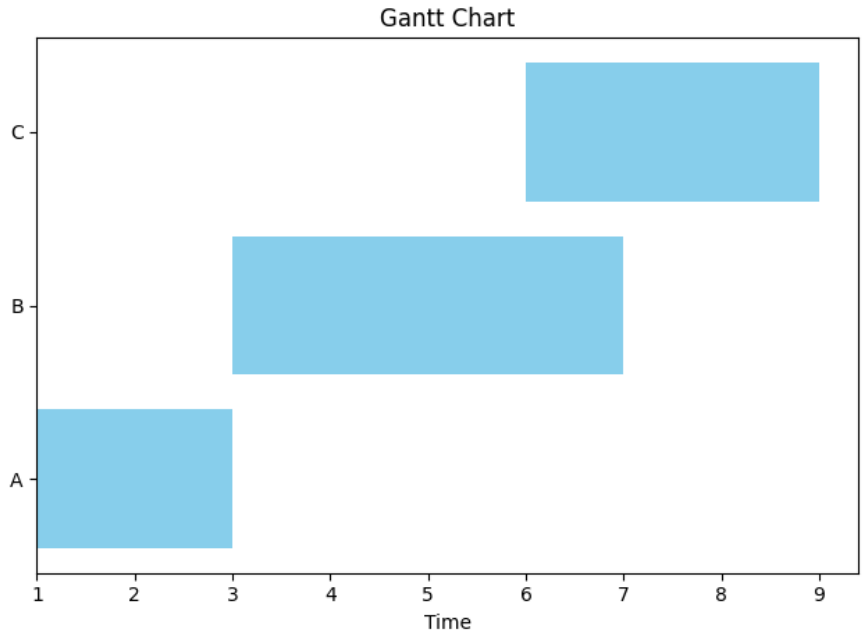
- Show progress toward goals clearly and quickly.
- Compare actual performance with targets.
- Communicate key metrics in dashboards or reports.



Gantt Chart: A Gantt chart uses horizontal bars to show project tasks over time. It helps in planning, scheduling and tracking progress visually.

When to use:

- Plan and schedule tasks with dependencies.
- Track progress and resource use over time.
- Communicate timelines and milestones clearly.

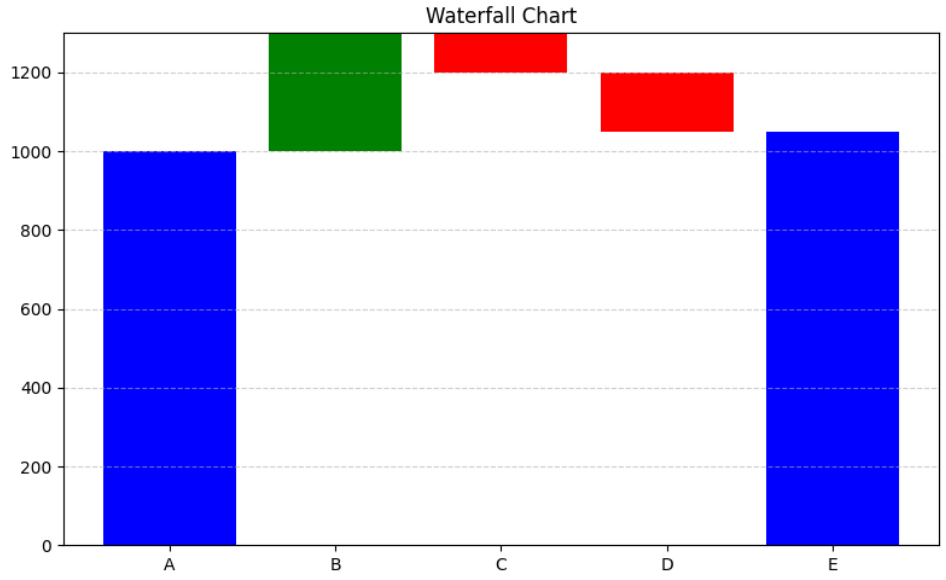


Waterfall Chart

- A waterfall chart shows how positive and negative values affect a total over time. It's useful for tracking changes in financial data, budgets or performance metrics.

When to Use:

- Analyze step-by-step gains or losses over time.
- Understand the impact of different factors on totals.
- Present complex changes clearly and visually.



Thank You...